Mentimeter

Laborator 11

Genericitate

Mentimeter

Instructions

1 1

Mentimeter

```java
public static void main(String[] args) {
    List myList = new LinkedList();
    myList.add(new Integer( value: 0));
    myList.add(new String( original: "ala bala"));
    for (Object x : myList) {
        Integer number = (Integer) x;
        System.out.println(number);
    }
}
```

Problemos Raw Type

Mentimeter

De ce nu e ok Raw Type?

- cod greu de citit (prea multe cast-uri)
- erori de executie (pun String in loc de Integer)

1

Mentimeter

```java
public static void main(String[] args) {
    List<Integer> myList = new LinkedList<Integer>();
    myList.add(new Integer( value: 0));
    myList.add(new String( original: "ala bala"));

    for (Integer number : myList) {
        System.out.println(number);
    }
}
```

Solutie

Mentimeter

Solutie

- restrictionam tipul obiectelor (pot pune numai Integer)
- daca nu **erori de compilare, nu de runtime**

1 1

Mentimeter

Genericitatea

- imbunatateste lizibilitatea codului
- creste gradul de robustete

1

Mentimeter

Interface List<E>

Type Parameters:

`E - the type of elements in this list`

All Superinterfaces:

`Collection<E>, Iterable<E>`

All Known Implementing Classes:

`AbstractList, AbstractSequentialList, ArrayList, AttributeList, CopyOnWriteArrayList, LinkedList, RoleList, RoleUnresolvedList, Stack, Vector`

Notatie

Mentimeter

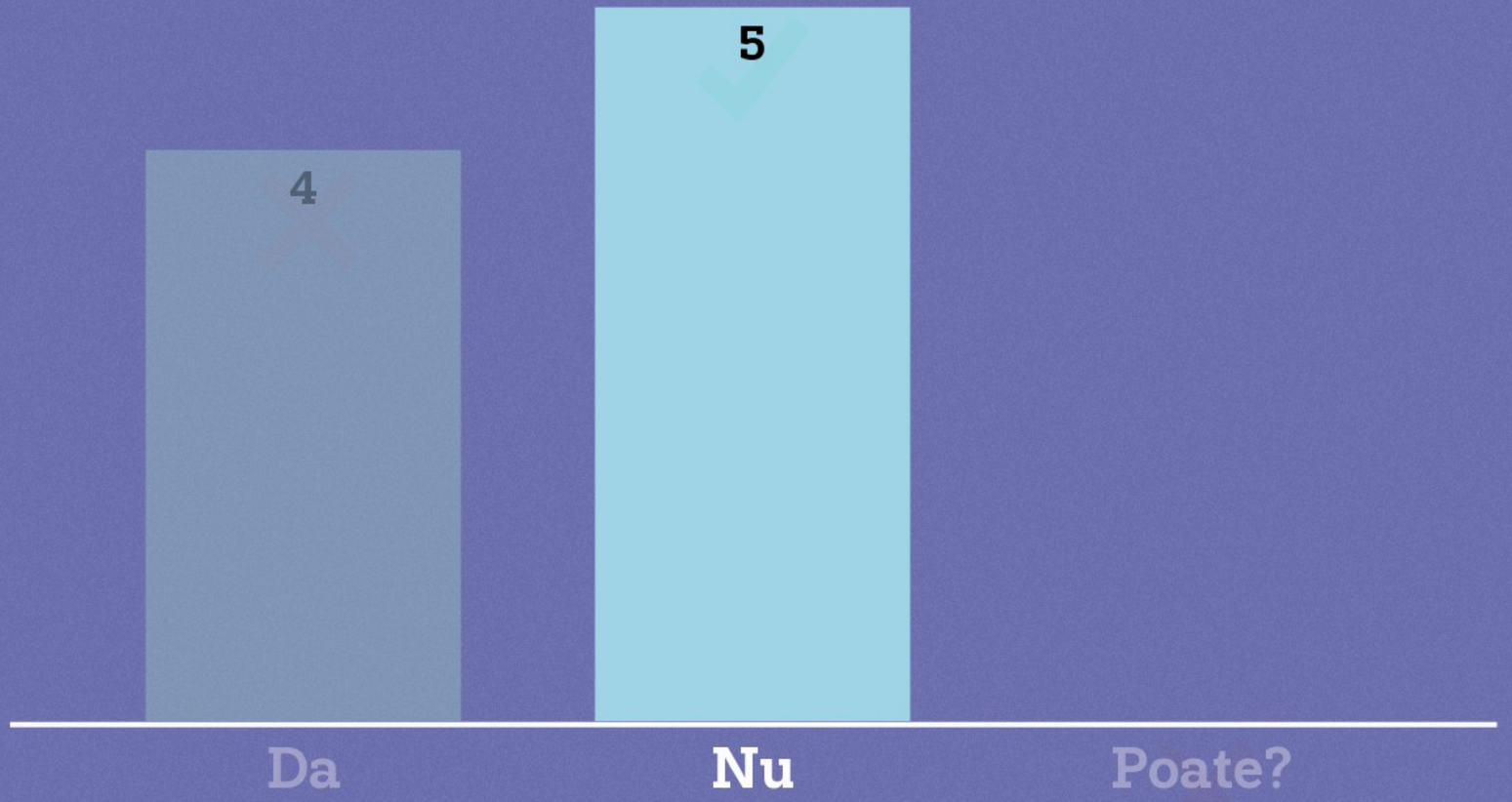
```
List<Integer> myList = new LinkedList<Integer>();
```

Instantiere

3 1

E ok a 2-a linie?

Mentimeter



9

Leaderboard

Mentimeter

711 p	🎱 Victor *fastest*
648 p	🐛 Dr Centipede
623 p	🐸 Mari
613 p	⛄ Frosty
605 p	🍀 Maria
0 p	🐺 Bogdan
0 p	🤔 Mada
0 p	💍 p5 202 E
0 p	😸 P5202I

9

Mentimeter

Nope

daca as face "cast"-area de la List<String>
la List<Object> atunci ajung la aceleasi
buguri ca la inceput

Mentimeter



Generic<Parent> nu e subtip de la Generic<Child>

2 2

Mentimeter

Wildcards

- avem aceeasi implementare indiferent de tip
- nu putem folosi Collection<Object> pentru ca Collection<T> nu ar fi subtip pentru Collection<Object>

1

Mentimeter

```
public static void show(Collection<Object> collection) {
    for (Object obj : collection) {
        System.out.println(obj);
    }
}

public static void main(String[] args) {
    Collection<Integer> list = new ArrayList<>();
    Main.show(list);
}
```

Not okay

1

Mentimeter

```java
public static void show(Collection<?> collection) {
  for (Object obj : collection) {
    System.out.println(obj);
  }
}

public static void main(String[] args) {
  Collection<Integer> list = List.of(1);
  Set<Integer> hashSet = Set.of(1, 5);

  Main.show(list);
  Main.show(hashSet);
}
```

Okay

1 1

Mentimeter

Bounded wildcards



Mentimeter

```java
class Human {

  private final String name;

  public Human(String name) {
    this.name = name;
  }

  public static void add(Collection<?> students, Human s) {
    ((Collection<Human>) students).add(s);
  }
}
```

Exemplu 1

Mentimeter

```java
class Student extends Human {

  private final String university;

  public Student(String name, String university) {
    super(name);
    this.university = university;
  }
}
```

Exemplu 2

Mentimeter

```
public static void main(String[] args) {
    Collection<Human> humans = List.of(new Human( name: "of"), new Human( name: "of2"));
    Collection<Student> students = List.of(
        new Student( name: "ceva",  university: "Uni"), new Student( name: "ceva1",  university: "Uni"));

    Human.add(humans, new Human( name: "of2"));
    Human.add(students, new Student( name: "ceva2",  university: "Uni"));
```

Exemplu 3



Mentimeter

De ce nu pot adauga direct?

- wildcard nu specifica niciun tip anume
- nu pot adauga intr-o colectie decat elemente de un anumit tip
- pot pune numai null

Mentimeter

```java
public static void show(Collection<? extends Human> humans) {
    for (Human human : humans) {
        System.out.println(human);
    }
}
```

Solutie

Mentimeter

```
public static void add(Collection<? extends Human> students, Human s) {
    students.add(s);
}
```

Nu merge la orice

Mentimeter

De ce nu?

- Collection<? extends Human> => o colectie cu tipul unei clase care extinde Human
- pot sa am Collection<Student> si incerc sa adaug un Human
- **eroare de compilare**

1

Mentimeter

```java
public static void add(Collection<Human> students, Student s) {
    students.add(s);
}
```

Solutie

Mentimeter

Bounded Wildcards

- upper bounded wildcards: Collection<? extends Human> (toate clasele care extind Human sau Human)
- lower bounded wildcards: Collection<? super Student> (toate clasele care sunt superclasa pentru Human sau Human)

Mentimeter

```
public static void add(Collection<? super Human> students, Human s) {
  students.add(s);
}
```

Lower bounded wildcards

1

Mentimeter

De ce pot modifica aici?

- pot sa adaug un Student in Collection<? extends Human>
- un Student e un Human (un fel de upcasting)
- pot adauga numai Human sau un subtip de-al clasei Human

Mentimeter

```
public static void show(Collection<? super Human> humans) {
    for (Human human : humans) {
        System.out.println(human);
    }
}
```

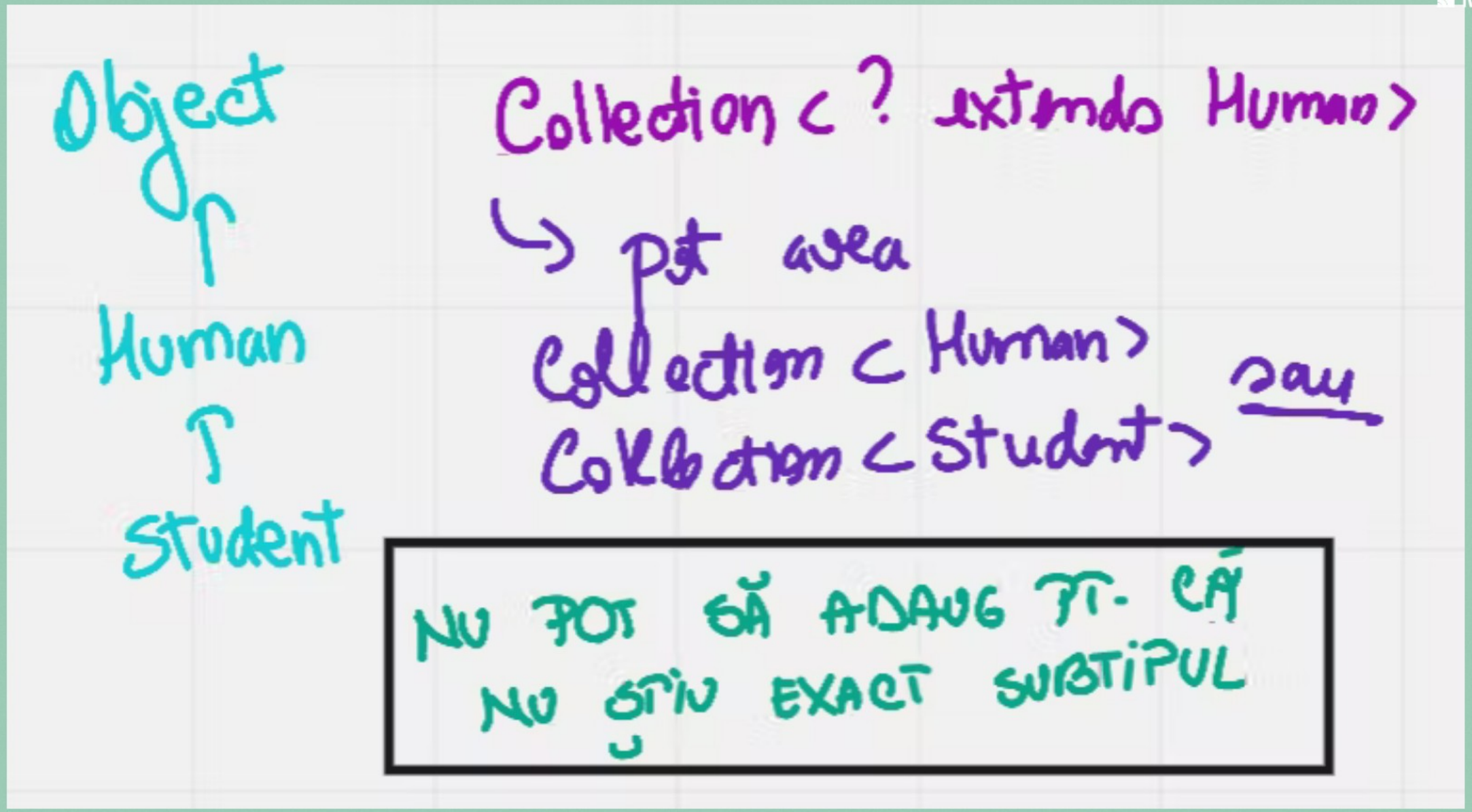
Nu mai pot parcurge colectia

Mentimeter

TL;DR

→ vreau sa parcurg colectia => ? extends T

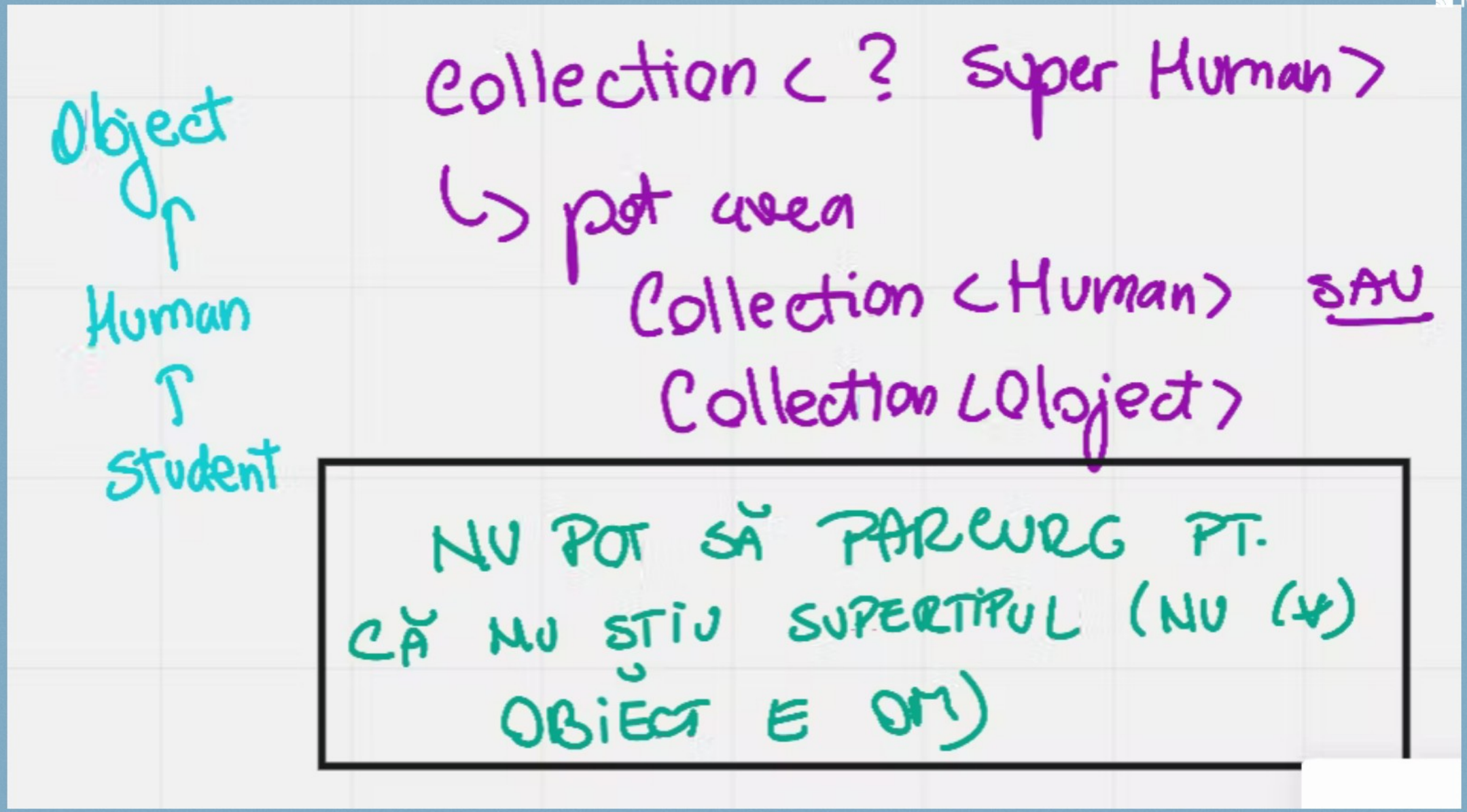
→ vreau sa modific colectia => ? super T



Explicatie 1

Mentimeter

Mentimeter



Explicatie 2

Mentimeter

Type Erasure

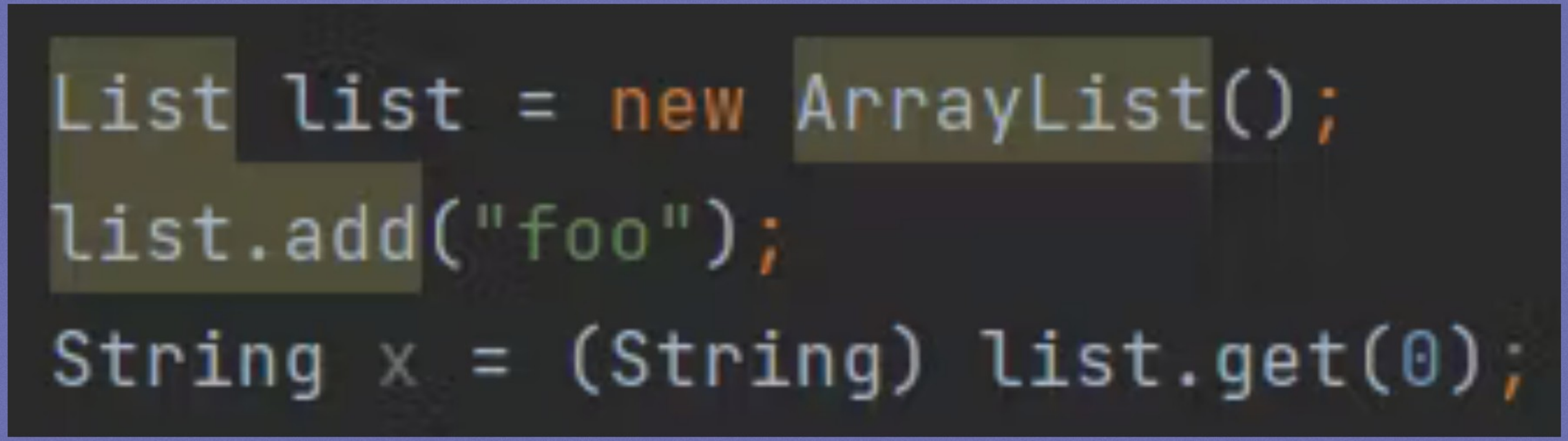
- am o clasa generica Generic<T>
- pot sa imi fac o instanta Generic generic fara sa ii dau un tip
- se ia automat Object daca nu e dat un tip

Mentimeter

```
List<String> list = new ArrayList<String>();
list.add("foo");
String x = list.get(0);
```

Initial

Mentimeter



Compilatorul face oricum inlocuirea cu raw type dupa (type erasure)

Mentimeter

```java
public static <T> void show(T[] list) {
    for (T obj : list) {
        System.out.println(obj);
    }
}
```

Metode generice

?

Mentimeter

```java
class MyCollection<T> {

  private final T[] values;

  public MyCollection(T[] values) {
    this.values = values;
  }

  public boolean contains(T value) {
    return Arrays.stream(values).anyMatch(v -> v.equals(value));
  }
}
```

Clase generice 1

1

Mentimeter

```java
public class Main {

  public static void main(String[] args) {
    Integer[] integers = {1, 2, 3};
    String[] strings = {"a", "b", "c"};

    MyCollection<Integer> integerCollection = new MyCollection<Integer>(integers);
    MyCollection<String> stringCollection = new MyCollection<String>(strings);

    System.out.println(integerCollection.contains(1));
    System.out.println(stringCollection.contains("b"));
  }
}
```

Clase generice 2

4

Mentimeter

Demo